

Report 5/19/26

Anthony Kuang

1 Summary

2 Proposal

Grounding executable visual semantics in unified multimodal models

2.1 Artifacts

Datasets will be released on Hugging Face, Code will be on a personal fork of BAGEL.

2.2 Abstract

VLMs that emit graphics code (TikZ, manim, SVG, matplotlib, slide DSLs) treat the program purely as text: the model never observes what its own code renders to, and any signal about visual fidelity must arrive through a text-only proxy. Unified models — architectures that natively generate both text and image tokens in a single model — remove this restriction, since the rendered output can sit in the same training sequence as the program that produced it. We propose to test whether a *continued mid-training* stage on (code, rendered image) sequences, initialized from the released BAGEL 7B checkpoint, improves a unified model’s ability to (i) generate code that renders to a target image, (ii) predict what a given program will render to, and (iii) reason about images by referring to their underlying executable code structure (e.g., using a program as an intermediate representation when inputting an image). The mid-training corpus mixes prompt-conditioned code-image pairs (`<prompt><code><image>`), render traces (e.g. `<prompt><code chunk 1: draw (0,0) circle (1cm);><image A: one circle><code chunk 2: draw (2,0) circle (1cm);><image B: two circles> <code chunk 3: node at (1,0) hello;><image C: two circles + hello>`), and prompt-free `<code>`, `<image>` & `<image>`, `<code>` pairs. We use TikZ as the primary domain for its determinism and existing data scale, extend to SVG, matplotlib, manim, and slide DSLs to test cross-DSL transfer, and follow mid-training with a renderer-as-verifier RL stage. Throughout, we compare against matched-compute baselines: the same BAGEL checkpoint continued on the same code, but with the loss on rendered images masked out, so the model sees the same data but receives no supervision on producing images.

2.3 Introduction

Code that renders to images is an unusual modality: it is symbolic and discrete, yet it has a deterministic visual semantics given a renderer. Standard VLMs and code LMs handle the symbolic side well but never see the rendered output during training. This creates a predictable failure mode where the model is rewarded for syntactically and stylistically plausible code, not for code that produces the intended image. Recent benchmarks (SGP-Bench, VIGA, AutoPresent) document exactly this gap: models can write code that looks right and runs, but the rendered output frequently diverges from the prompt’s visual intent. Unified models offer a structural fix. Architectures such as Chameleon, Transfusion, and BAGEL process image and text tokens within a single transformer, with a single optimizer and shared global attention. An interleaved sequence `<code> ...</code> <image> ...</image>` is a first-class training example. The question this proposal asks is whether explicitly training on such sequences before any task-specific fine-tuning provides the capability to ground code in its visual semantics, and whether this capability transfers across rendering languages and downstream tasks.

We frame this as a **mid-training** intervention rather than SFT or RL for three reasons. First, mid-training data can be procedurally generated at scale (we control the renderer), avoiding the data scarcity that limits SFT. Second, mid-training shapes representations before alignment, which is the right stage for installing a new modality-bridging capability rather than steering existing ones. Third, mid-training composes with RL: a model that has seen many (`, image`) pairs has a much better initialization for renderer-based reward.

Hypotheses. We will evaluate 5 claims:

1. **H1 (grounding).** Mid-training on (`, image`) pairs improves code-to-image generation over a matched-compute baseline that sees the same code as text-only data.
2. **H2 (understanding).** The same intervention improves code-to-image *understanding* (predicting renders, answering questions about programs via their visual output).
3. **H3 (transfer).** A model mid-trained on one DSL (TikZ) transfers to held-out DSLs (SVG, matplotlib) compared to a DSL-matched text-only baseline.
4. **H4 (RL initialization).** Mid-training improves the sample efficiency and final performance of a subsequent renderer-as-verifier RL stage.
5. **H5 (context unrolling).** A mid-trained model can use its own rendered intermediate outputs as context for subsequent code generation, producing better final code than single-shot generation. This is a natural test of Omni’s context unrolling findings.

Contributions. (1) A controlled study of (`, image`) mid-training in unified models, with matched-compute baselines for SFT-only, pretraining-mixture, and text-only-code conditions. (2) A curated multi-DSL training corpus with deterministic renderers and decontamination against public benchmarks. (3) An evaluation suite separating generation and understanding capabilities and probing cross-DSL transfer. (4) An RL stage with renderer-grounded reward, isolating the contribution of mid-training to downstream RL.

2.4 Related Works

Code generation with execution feedback. CodeRL and Self-Debugging use program outputs (test results, error traces) as a training or inference-time signal for general-purpose code. Code World Model extends this to learning to predict execution state (where it treats the variables as the state and code as actions). None of this line of work treats the *rendered image* as the execution result, and none operates in a unified-modality model where the image is in-distribution for the generator.

Code generation for images. VIGA closes an analysis-by-synthesis loop at inference, using a VLM to edit code and re-render. AutoPresent generates slide programs and evaluates the resulting decks. SGP-Bench basically probes whether models understand what graphics programs render to. Design2Code and WebSight generate HTML/CSS from screenshots. All operate on standard (non-unified) backbones; the hypothesis here is that a unified backbone with shared attention between the code and the rendered images changes what training signal is available, not merely the inference loop.

Unified models. Chameleon tokenizes images into VQ-VAE codes and trains a single autoregressive transformer over text+image tokens; this works, but VQ tokenization degrades high-frequency fidelity. Transfusion replaces the image side with diffusion over continuous VAE latents, with a joint $\mathcal{L}_{\text{LM}} + \lambda \mathcal{L}_{\text{DDPM}}$ objective, and performs better. Mixture-of-Transformers decouples non-embedding parameters per modality while preserving global attention, matching dense Transfusion performance in fewer steps. BAGEL provides an open-weights instantiation of the MoT family at scale.

Mid-training. Recent open-weights releases (BAGEL, Emu3, Phi-family models) describe staged training pipelines with curated mid-training data that install specific capabilities (long context, code, math, multimodality) between pretraining and post-training.

2.5 Data Collection

Primary domain: TikZ. TikZ is a pretty clean starting point: (a) deterministic renderer (`pdflatex` $\rightarrow$ `pdf2png`), (b) compact programs, (c) substantial existing corpora (DaTikZ, DaTikZv2, arXiv-mined TikZ), and (d) coverage of a wide range of visual primitives (diagrams, plots, geometric figures).

- Aggregate DaTikZv2 plus a selected scrape of arXiv-source TikZ, deduplicated by AST hash.
- Render every program with a sandboxed `pdflatex` pipeline, discarding non-compiling and timeout cases.
- Filter for image diversity using CLIP-embedding nearest-neighbor deduplication; prevent contamination.

Secondary domains. For cross-DSL transfer (H3) we need additional domains with renderers as deterministic as TikZ. We include four:

- SVG: Wikimedia Commons plus the SVG-Stack corpus, rendered with `resvg`.
- Matplotlib: Stack Overflow accepted answers tagged matplotlib
- Manim: scraping public manim repositories (3Blue1Brown repos).
- Slide DSLs: apply AutoPresent’s data construction pipeline, targeting python-pptx as the DSL.
- These domains are held out from mid-training in the cross-DSL transfer condition (A5) and seen in the multi-DSL condition.

Sequence format mix. Within each DSL, we split the corpus three ways. Around 70% goes to prompt-conditioned full-program pairs (`<prompt><code><image>`), which is the main format and supplies most of the scale. Roughly 20% goes to render traces (`<prompt><code chunk 1><image 1><code chunk 2><image 2>...`), where the program is split into successive chunks and the partial render is interleaved after each. Chunking is DSL-specific: after each top-level `\draw` or `\node` for TikZ, after each plotting call for matplotlib, and after each immediate child of the root `<svg>` for SVG. Render traces give the model fine-grained credit assignment between local code edits and local image changes (the visual analog of Code World Model’s per-line execution traces). The remaining 10% goes to prompt-free pairs, half `<code><image>` and half `<image><code>`, which prevents the model from learning to ignore the code when a prompt is available, and trains the inverse-rendering direction needed for the "code as intermediate for image reasoning" capability. Ablate data split in A4.

Synthetic augmentation. For each DSL, I will also build a procedural generator that samples programs (parameterizing primitives, colors, layouts, text labels). This gives us (a) more data scale, (b) controlled difficulty, and (c) a clean separation between "natural" code (human-written, stylistically varied) and "synthetic" code (grammar-sampled, uniformly distributed). Could ablate natural vs. synthetic.

Targets and scale. The mid-training stage will involve ~ 50 B tokens of (`code`, `image`) sequences: 20B from TikZ (real + synthetic, ~ 5 M programs on average), 10B from SVG, 10B from matplotlib, and 10B from others (Graphviz, manim, slide DSLs, blender). On the image side, BAGEL uses the FLUX VAE (8x downsampling, 16-channel latent) with a 2×2 patch embedding, giving ~ 256 latent tokens per 256×256 image. Each image span carries BAGEL’s 3-token-set representation: noised VAE (for the rectified flow loss), clean VAE (as context), and ViT tokens (also context). Sequences are formatted as `<text prompt><code>...</code><B0I><ViT tokens><clean VAE><noised VAE><EOI>`, with the text prompt either a real caption (when available) or a VLM-generated description. Note: trace sequences will inflate token count substantially: a 50-line TikZ program with 50 intermediate renders adds 50 image spans of 256 tokens each, so trace examples are much longer than full-program examples.

Decontamination. All public eval sets are removed by exact match and CLIP-image similarity within some radius r .

2.6 Mid Training

Backbone. BAGEL 7B active / 14B total, initialized from the released checkpoint. Inherit BAGEL’s design: Qwen2.5 LLM initialization for both experts, SigLIP2 ViT for the understanding side, FLUX VAE for the generation side, QK-Norm, rectified flow on the image side, and generalized causal attention with the clean/noised/ViT 3-token-set per image. Code routes through the understanding expert in the primary condition; ablation A2 tests the generation-expert alternative.

Mid-training stage. Continue training BAGEL for 50B tokens on the corpus described above. Use the same BAGEL joint loss.

Direction conditioning. BAGEL’s generalized causal attention is direction-agnostic between splits, so we mix sequence orderings. The full-program and trace formats default to `prompt to code to image`. The 10% prompt-free slice splits evenly between `code to image` and `image to code`. Ablate in A4.

Baselines matched compute. All baselines start from the same released BAGEL checkpoint. The primary baseline continues training BAGEL on the same code corpus with the rectified flow loss on noised VAE tokens set to zero. The model still sees the images in context (clean VAE and ViT tokens remain) but is not supervised on producing them. This isolates the contribution of image-side supervision from the contribution of simply seeing more code. A second baseline folds the same (`code`, `image`) data into pretraining rather than as a separate mid-training stage, to test whether staging itself matters.

2.7 RL

After mid-training and a short SFT pass on instruction-following code-to-image data, we apply a render-as-verifier RL stage. The reward signal combines (i) a binary compile/render check, (ii) a CLIP image-image similarity score between the rendered output and the target image, and probably some other reward model that penalizes the failures that CLIP is known to underweight (text fidelity, fine geometry, high-frequency components). GRPO.

2.8 Ablations

A1: Where does mid-training help? Split evals into generation (write code for an image), understanding (predict what code renders to), and joint (interleaved reasoning, e.g. "modify this program so the circle is red" given code and image).

A2: Modality routing in MoT. (i) code routes through the understanding expert (default), (ii) code routes through the generation expert. This directly tests whether code, while syntactically text, is representationally closer to image or text in a unified model. Compare representations with PCA.

A3: Natural vs synthetic code. 3 mid-training mixtures at matched token counts: 100% natural, 100% synthetic, and 50/50. Tests whether human stylistic variation matters or whether grammar-sampled coverage is enough.

A4: Sequence direction. Code→image only vs. image→code only vs. mixed. Hypothesis: mixed wins on understanding evals because image→code forces the model to use the image as input; code→image-only models will likely treat the image side as output-only and underperform on understanding.

A5: Cross-DSL transfer. Mid-train on TikZ only, evaluate on SVG and matplotlib generation/understanding. Compare against (a) the text-only TikZ baseline and (b) mid-training on all other DSLs. Measures how much of the gain is DSL-specific vs. a general code-image grounding capability.

Future ablation: scale. BAGEL is released only at 7B active. A proper scale ablation (does the mid-training gain grow, shrink, or hold with model size?) would require retraining smaller unified models from scratch.

3 Notes

The core idea is that models can generate code that renders into images, such as TikZ code, graphics code, or slide-generation code. However, standard VLMs typically generate the code only at the language level and do not model or understand the execution result of that code. Unified models can handle both language generation and image generation. This opens up the possibility of training unified models on (`, image`) pairs as a mid-training objective, and then studying whether this improves more general **code-to-image generation and understanding capabilities**.

Objectives: So basically, unified models can generate code and understand its execution result (the rendered image), so training on (`, image`) pairs as mid-training should improve code-to-image abilities. There are several hypotheses which I'm not entirely convinced of, so would need to cover in the literature review. 1) standard VLMs treat code as text and never "see" what it renders (reasonable hypothesis because early fusion vs late fusion). 2) mid-training vs. SFT, RL, or pretraining is the right intervention strategy (why not just SFT with images).

3.1 Code Generation

VIGA

1. Helmholtz framed perception in the inverse problem
2. Roberts made it computational by using a generative model (toy domains)
3. Analysis-by-synthesis
4. Bayesian/probabilistic framing, but slow inference and brittle priors
5. Differentiable rendering made things faster but couldn't escape local minima like if the initial rendering positions were wrong
6. VLM-based approaches like **VIGA**: *broad semantic priors* and *discrete reasoning* (but lack fine-grained pixel grounding, hence the loop).

3.2 Unified Models

Watched Luke Zettlemoyer's talk on intro to Mixed-modal Language Modeling.

Goal: Single, simple architecture for all modalities (mostly images and text for now). Mixed-modal generation. Interleaved text and images in output.

Background: Image Generation

Image generation in unified models typically use one of two standard image generation approaches:

1. Token-based / autoregressive (Parti style; a VQ-VAE encodes an image into discrete tokens, an autoregressive model generates those tokens in raster order, and the VQ-VAE decoder converts them into an image)
2. Diffusion (Dall-E 2 style; specifically, latent diffusion models use a VAE-style continuous encoder to compress images into latent space, where a diffusion/flow matching model learns to iteratively denoise and generate samples before the decoder reconstructs the final image)

Background: Early-fusion vs Late-fusion

- **Early-fusion** approach, where all modalities are projected into a shared representational space from the start. But presents significant technical challenges in training stability.

Architectures

Chameleon

- Tokenize everything / autoregressive approach
- **Image tokenizer:** We train a new image tokenizer based on Gafni et al. (2022), which encodes a 512×512 image into 1024 discrete tokens from a codebook of size 8192. For training this tokenizer, we use only licensed images. Given the importance of generating human faces, we up-sample the percentage of images with faces during pre-training by 2 times. A core weakness of our tokenizer is in reconstructing images with a large amount of text, therefore upper bounding the capability of our models, when it comes to heavy OCR-related tasks.
 - Why VQ-VAE tokens bad for OCR related? VQ-VAE tokenizers perform poorly on OCR because their coarse, perceptual, and spatially compressed discrete codes lose the fine-grained, high-frequency structure needed to represent exact character shapes and precise text composition.
- **Pre-training data:** 2 stages 80/20 split, 1. text-only, 2. text-image (For all Text-To-Image pairs we rotate so that 50% of the time the image comes before the text i.e., captioning), 3. text/image interleaved

Transfusion

- Unified model with two losses on shared transformer / autoregressive for text, diffusion for images
- **Image representation: Chameleon** discrete representation loses information. Use continuous representations with pretrained VAE, encodes a 256×256 image into a $32 \times 32 \times 8$ latent grid.
 - **Patch encoder/decoder:** Compresses image representations into vectors fed into transformer (and back). Two variants, simple linear projection, or U-Net down/up blocks.
- **Sequence to feed into transformer:** Text tokens embedded normally. Images noised at sampled timestep t , patchified, projected to $\mathbb{R}^d$, with timestep embedding added pre projection. Surrounded by BOI/EOI markers and spliced into the text sequence.
- Causal attention mask between text, bidirectional within each image’s patches.
- $\mathcal{L}_{\text{Transfusion}} = \mathcal{L}_{\text{LM}} + \lambda \cdot \mathcal{L}_{\text{DDPM}}$ with $\lambda = 5$.
- **DDPM review:**
 - **Forward process:** Choose T . Define forward process with noise schedule going up to T timesteps: $q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{(1-\beta_t)} \cdot x_{t-1}, \beta_t \cdot I)$. Typically, β_1 is tiny (10^{-4}) and β_T is bigger (0.02), with $T = 1000$. x_T becomes pure noise.
 - **Closed form of forward process:** Unroll the Markov Chain to get $q(x_t|x_0)$, uses α_t which is an accumulation of β_t terms.
 - **Reverse process:** We want the reverse $q(x_{t-1}|x_t)$. Not tractable; depends on unknown data distribution. But when step size is small, can model as approximately Gaussian: $p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$. In practice we fix Σ (can use upper bound of forward variance β_t which is reasonable for small step size) and predict only μ_θ .
 - **Reparameterization** So predicting μ_θ works, but not ideal because the mean of small timesteps T is we can reparameterize to modeling $\epsilon_\theta(x_t, t)$ instead.
$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \cdot (x_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \cdot \epsilon_\theta(x_t, t))$$

- **Diffusion in Transfusion:**

- Take an interleaved sequence. The text is tokenized into integers. For each image, do the following: Sample t . The image is encoded by a VAE into a continuous latent grid, which is then noised into a noisy latent grid x_t , which is then patchified into a sequence of continuous vectors. Add timestep embedding to each patch. Project each patch to $\mathbb{R}^d$ via linear or U-Net down blocks. Splice the sequence between BOI/EOI text tokens.
- Attention mask is causal for text positions, extended to bidirectional within each image’s patches. Text positions output logits over the next token (cross-entropy loss with the true next token). Image positions output ϵ_θ for that patch. collectively, the per-patch outputs assemble into $\epsilon_\theta(x_t, t, c)$, the model’s joint prediction of the noise vector for the whole image, where x_t is the noisy latent, t is the timestep (signaled via the timestep embedding added to each patch), and c is the causal context (all prior text and prior images, but not future material outside this image, bidirectional attention is contained within each image’s patch range).

- **Inference:** Decoder has 2 different modes. Autoregressive text sampling until BOI; then append n noise-filled patches and run 250 denoising steps (transformer forward + DDPM update, overwriting patches in-place each step); append EOI; resume text mode. Text prefix KV cache is reusable across denoising steps, image patches are not.

- **Pretraining data:** 0.5T tokens at 1:1 text/image ratio for controlled experiments; scaled to 2T tokens (1T text + 692M image-caption pairs, 5 epochs) for the flagship 7B model. Caption-first 80% of the time, image-first 20%.

- **versus Chameleon:**

- Image generation: $\sim 34\times$ less compute to match FID; $\sim 2\times$ lower FID at matched FLOPs.
- Image-to-text: matches Chameleon at 21.8% of FLOPs.
- Text-only: also better (kinda surprising), 50-60% of Chameleon’s FLOPs for matched perplexity. Hypothesis: less output-distribution competition when image generation isn’t sharing the softmax, or diffusion is more parameter-efficient for images so more capacity available for text.

Mixture-of-Transformers (MoT)

- Sparse multi-modal transformer that decouples **all** non-embedding parameters by modality (text/image/speech) while keeping global self-attention across the full sequence. Drop-in replacement for dense transformer in Chameleon/Transfusion-style training.
- **Motivation:** PCA of dense multi-modal model activations shows modalities cluster into distinct regions of feature space across all layers / training checkpoints, despite being processed as uniform tokens. Modalities also exhibit conflicting training dynamics in dense models. Suggests modality-specific parameters are natural.
- **Architecture:** For each token x_i of modality m_i , apply modality-specific weights:
 - Modality-specific Q/K/V/O projections: $W_Q^{m_i}, W_K^{m_i}, W_V^{m_i}, W_O^{m_i}$
 - Modality-specific FFN: FFN^{m_i}
 - Modality-specific LayerNorms (attn + fnn)
 - **Shared:** global self-attention computed across all tokens after modality-specific projections
- **Computation flow:** Group tokens by modality $\rightarrow$ modality-specific projections $\rightarrow$ restore sequence order $\rightarrow$ global self-attention $\rightarrow$ modality-specific output projection + LN + FFN + LN.
- Same FLOPs as dense (each token still goes through one FFN/attn), but more total parameters (one set per modality). Distinct from MoE: routing is deterministic by modality (no learned router, no load balancing), and decoupling extends to attention + LN, not just FFN.

- **Contrast with MoE-4x baseline:** MoE-4x uses Expert Choice routing on the FFN. EC violates causality, so at inference they re-use EC routing on validation data (acknowledged as confound - router can peek at future tokens, and distribution shift hurts expert balance).
- **Key results** (all FLOPs-controlled, from scratch):
 - **Chameleon 7B** (AR text + AR image): Matches dense final loss in 55.8% of FLOPs overall; image modality reaches dense loss in 34.8% of steps. MoE-4x’s image gains vanish by 7B; MoT’s gains hold or grow across scales (37M→7B).
 - **Chameleon+Speech 7B:** Speech modality matches dense in 22.9% of training steps; gains preserved on image/text.
 - **Transfusion** (AR text + diffusion image): 760M MoT beats 1.4B dense on CLIP (0.214 vs 0.206), FID (21.1 vs 24.7), CIDEr (0.320 vs 0.286) - using half the FLOPs. Image training loss matched in ~30% of dense steps.
 - **Transfusion text:** Improvement marginal (vs strong gains in Chameleon). Authors hypothesize the separate diffusion objective already decouples text/image gradients, so MoT’s parameter decoupling is partly redundant for text here.
- **Wall-clock** (7B, 256 A100s): MoT matches dense image quality in 47.2% of wall-clock time, text in 75.6%. MoE-4x is $1.7\times$ slower than dense on image (routing overhead). Gains grow with GPU count (16→256): image step ratio drops from 42.1% to 21.6%, since lower parameter-per-FLOP ratio reduces communication overhead.
- **Ablation, which components matter:** Untying FFN alone (à la MoMa) gives most of the gains. Adding Q/K/V untying yields further ~33% FLOPs saving on image, ~10% on text. Untying Layer-Norms on top is negligible. FFN matters most because it dominates FLOPs at 4k context and acts as modality-specific memory.
- **Leave-one-out analysis:** Three-tower MoT > any two-tower variant > one-tower dense. Effects are non-reciprocal: e.g. merging text+speech hurts both, but merging text+image leaves image largely intact while tanking text/speech. Suggests asymmetric modality competition.
- **Hybrid MoT + MoE:** Replacing MoT’s text FFN with a 4-expert MoE layer ("MoT + Text MoE-4x") further accelerates text without harming image, best of both in Chameleon. In Transfusion the text MoE training-loss gains don’t transfer to validation (router generalization issue with continuous image tokens).
- **Relevance to our proposal:** MoT is a natural backbone candidate. (code, image) pairs would route through the image+text towers with shared global attention, image tower learns the rendered output, text tower learns the program. Decoupled parameters might help since code and natural language likely have different optimal representations (cf. modality clustering in Fig 2). Open question: would code tokens want their own tower, or share with text?

BAGEL

Understanding and generation expert

OMNI